

Lecture 5 - Cascades

CASCADES OPTIMIZER

17

Object-oriented implementation of the previous Volcano query optimizer.

- **Top-down approach** (backward chaining) using branch-and-bound search.
- Depth-first search ordering of tasks via a stack.

Supports expression re-writing through a direct mapping function rather than an exhaustive search.



Graefe

CASCADES: KEY IDEAS

Optimization tasks as data structures.

- Patterns to match + transformation rule to apply

Rules to place property enforcers.

- Ensures the optimizer generates correct plans.

Ordering of moves by promise.

- Dynamic task priorities to find optimal plan more quickly.

Unified representation of rules & operators.

- Single search engine for logical and physical operators.

CASCADES: EXPRESSIONS

An **expression** represents some operation in the query with zero or more input expressions.

→ Optimizer needs to quickly determine whether two expressions are equivalent.

```
SELECT * FROM A
JOIN B ON A.id = B.id
JOIN C ON C.id = A.id;
```

Logical Expression: $(A \bowtie B) \bowtie C$

Physical Expression: $(A_{Seq} \bowtie_{HJ} B_{Seq}) \bowtie_{NL} C_{Idx}$

CASCADES: GROUPS

A **group** is a set of logically equivalent logical and physical expressions that produce the same output.

→ All logical forms of an expression.

→ All physical expressions derived from selecting allowable physical operators for the corresponding logical forms.

Group

Output:
{ABC}

Properties:
None

Logical Exprs

1. $(A \bowtie B) \bowtie C$
2. $(B \bowtie C) \bowtie A$
3. $(A \bowtie C) \bowtie B$
4. $A \bowtie (B \bowtie C)$
- ⋮

Physical Exprs

1. $(A_{Seq} \bowtie_{SM} B_{Seq}) \bowtie_{SM} C_{Seq}$
2. $(A_{Seq} \bowtie_{HJ} B_{Seq}) \bowtie_{HJ} C_{Seq}$
3. $(B_{Seq} \bowtie_{NL} C_{Seq}) \bowtie_{NL} A_{Seq}$
4. $(A_{Seq} \bowtie_{NL} C_{Seq}) \bowtie_{NL} B_{Seq}$
5. ⋮

CASCADES: GROUPS

A **group** is a set of logically equivalent logical and physical expressions that produce the same output.

- All logical forms of an expression.
- All physical expressions derived from selecting allowable physical operators for the corresponding logical forms.

<i>Group</i>	Output: {ABC} Properties: <i>None</i>	Logical Exprs	Physical Exprs	<i>Equivalent Expressions</i>
		1. $(A \bowtie B) \bowtie C$	1. $(A_{Seq} \bowtie_{SM} B_{Seq}) \bowtie_{SM} C_{Seq}$	
		2. $(B \bowtie C) \bowtie A$	2. $(A_{Seq} \bowtie_{HJ} B_{Seq}) \bowtie_{HJ} C_{Seq}$	
		3. $(A \bowtie C) \bowtie B$	3. $(B_{Seq} \bowtie_{NL} C_{Seq}) \bowtie_{NL} A_{Seq}$	
		4. $A \bowtie (B \bowtie C)$	4. $(A_{Seq} \bowtie_{NL} C_{Seq}) \bowtie_{NL} B_{Seq}$	
		⋮	5. ⋮	

CASCADES: GROUPS

A **group** is a set of logically equivalent logical and physical expressions that produce the same output.

- All logical forms of an expression.
- All physical expressions derived from selecting allowable physical operators for the corresponding logical forms.

	Output: {ABC} Properties: <i>None</i>	Logical Exprs	Physical Exprs
		1. $(A \bowtie B) \bowtie C$	1. $(A_{Seq} \bowtie_{SM} B_{Seq}) \bowtie_{SM} C_{Seq}$
		2. $(B \bowtie C) \bowtie A$	2. $(A_{Seq} \bowtie_{HJ} B_{Seq}) \bowtie_{HJ} C_{Seq}$
		3. $(A \bowtie C) \bowtie B$	3. $(B_{Seq} \bowtie_{NL} C_{Seq}) \bowtie_{NL} A_{Seq}$
		4. $A \bowtie (B \bowtie C)$	4. $(A_{Seq} \bowtie_{NL} C_{Seq}) \bowtie_{NL} B_{Seq}$
		⋮	5. ⋮

CASCADES: GROUPS

Instead of explicitly instantiating all possible expressions in a group, the optimizer implicitly represents redundant expressions in a group with a placeholder (e.g., **{ABC}**).

→ This reduces the number of transformations, storage overhead, and repeated cost estimations.

	Logical Exprs	Physical Exprs
Output: {ABC}	1. {AB} ⋈ {C}	1. {AB} ⋈ _{SM} {C}
	2. {BC} ⋈ {A}	2. {AB} ⋈ _{HJ} {C}
	3. {AC} ⋈ {B}	3. {AB} ⋈ _{NL} {C}
Properties: <i>None</i>	4. {A} ⋈ {BC}	4. {BC} ⋈ _{SM} {A}
	⋮	⋮

CASCADES: RULES

A **rule** is a transformation of an expression to a logically equivalent expression.

→ **Transformation Rule:** Logical to Logical

→ **Implementation Rule:** Logical to Physical

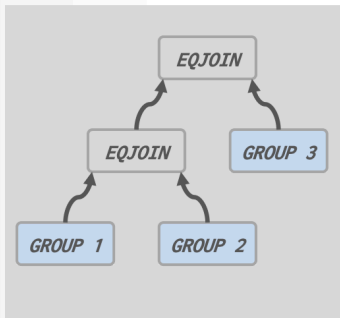
Each rule is represented as a pair of attributes:

→ **Pattern:** Defines the structure of the logical expression that can be applied to the rule.

→ **Substitute:** Defines the structure of the result after applying the rule.

CASCADES: RULES

Pattern



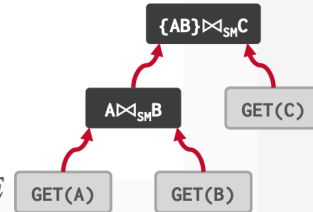
- Group
- Logical Expr
- Physical Expr

Transformation Rule Rotate Left-to-Right



Matching Plan

Implementation Rule EQJOIN → SORT MERGE



CASCADES: ENFORCERS

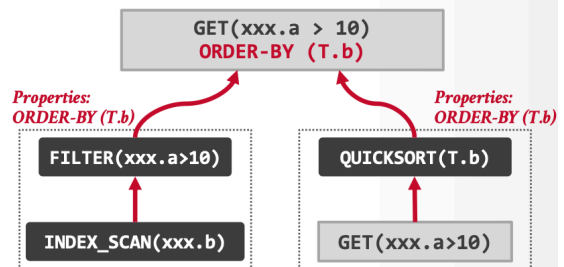
Represent enforcers as rules that change physical properties of a plan.

→ An enforcer inserts physical operators that change the physical properties of the plan.

Cascades does not require special casing to implement enforcers.

→ Volcano integrated enforcers as operators via ad hoc code.

```
SELECT * FROM xxx
WHERE xxx.a > 10 ORDER BY xxx.b;
```



CASCADES: TASKS

A **task** is a fine-grained unit of work that represents an operation in the query optimization process.

→ Break down optimization into smaller, manageable pieces to allow for more flexible and efficient exploration.

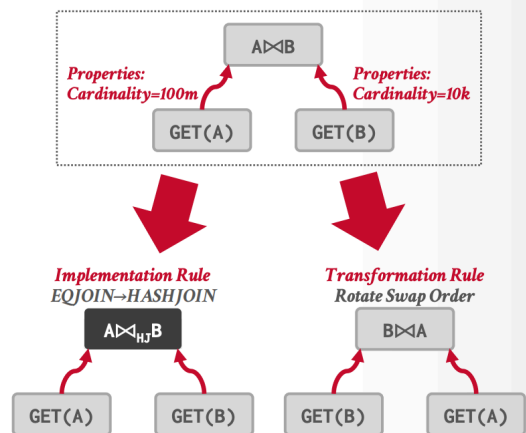
The order that a task is invoked can be customized via **promise**.

CASCADES: PROMISES

A task's **promise** is the estimated benefit of a move on a given expression relative to other tasks.

→ Example: Give a higher priority to a join order swap rule if the outer table expression is larger than inner table expression.

Optimizer must still ensure tasks execute in the order defined by their dependencies.

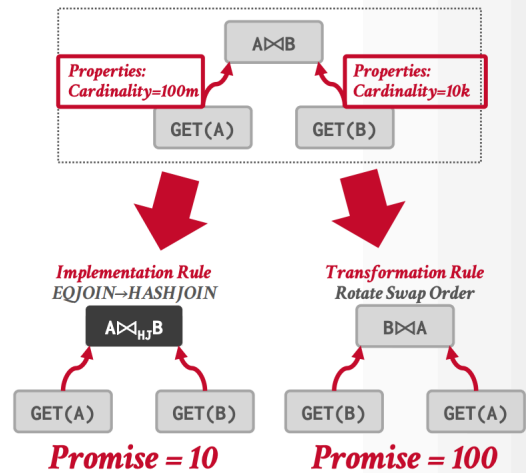


CASCADES: PROMISES

A task's **promise** is the estimated benefit of a move on a given expression relative to other tasks.

→ Example: Give a higher priority to a join order swap rule if the outer table expression is larger than inner table expression.

Optimizer must still ensure tasks execute in the order defined by their dependencies.

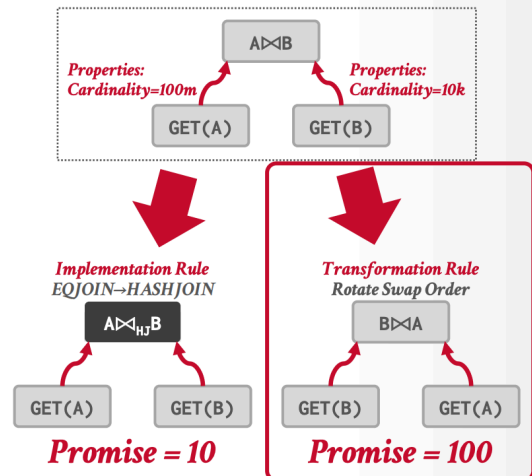


CASCADES: PROMISES

A task's **promise** is the estimated benefit of a move on a given expression relative to other tasks.

→ Example: Give a higher priority to a join order swap rule if the outer table expression is larger than inner table expression.

Optimizer must still ensure tasks execute in the order defined by their dependencies.



CASCADES: TASKS

#1 – Optimize Group:

→ Generate best physical plan for a group.

#2 – Optimize Expression:

→ Generate best physical plan for a specific expression.

#3 – Explore Group:

→ Generate logical expressions for a group.

#4 – Explore Expression:

→ Generate logical transformations for a specific expression.

#5 – Apply Rule:

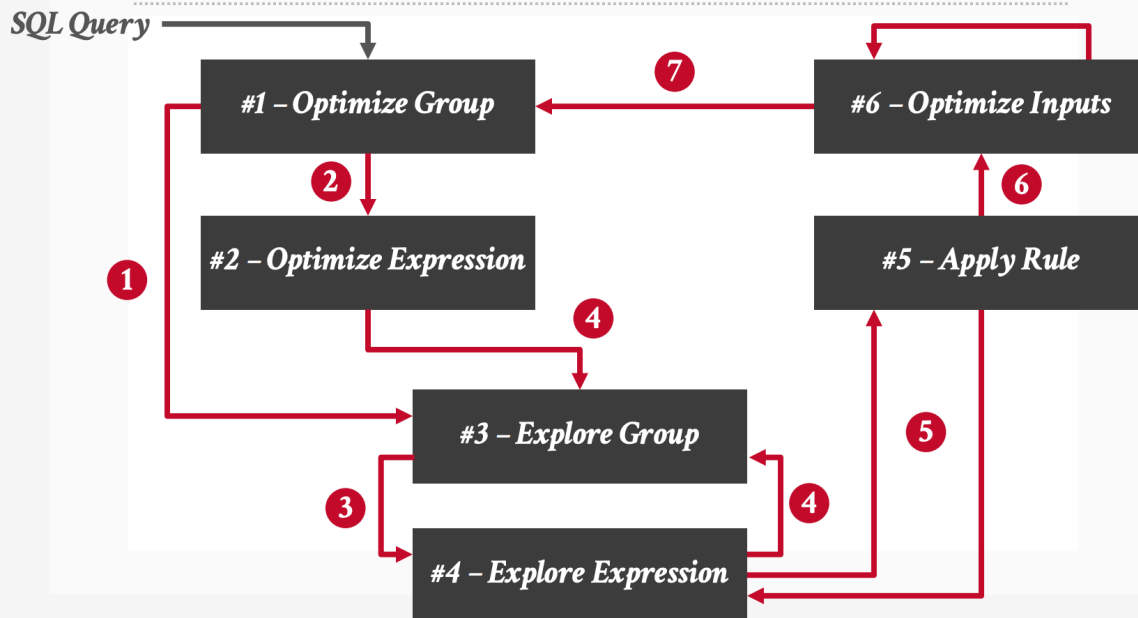
→ Apply a rule to an input expression.

#6 – Optimize Inputs:

→ Optimize the inputs of a given expression.

SPECIAL TOPICS (SPRING 2025)

CASCADES: TASK FLOW



SPECIAL TOPICS (SPRING 2025)

CASCADES: TASK-BASED SEARCH

Optimizer maintains a LIFO stack of tasks to perform actions on groups and expressions.

Stack ensures expressions are derived after the best plans of its input expressions are derived.

- Removes sequential ordering of independent optimizations.
- Tasks are stored in the heap rather than in the program stack to reduce OOM errors.

CASCADES: MEMO TABLE

Stores all previously explored alternatives in a compact graph structure / hash table.

Equivalent operator trees and their corresponding plans are stored together in groups.

Provides an overview of the optimizer's search progress that is used in multiple ways:

- Transformation Result Memorization
- Duplicate Group Detection
- Property + Cost Management
- Superfluous Rule Evaluation

PRINCIPLE OF OPTIMALITY

Every sub-plan of an optimal plan is itself optimal.

This allows the optimizer to restrict the search space to a smaller set of expressions.

→ The optimizer never has to consider a plan containing sub-plan **P1** that has a greater cost than equivalent plan **P2** with the same physical properties.

CASCADES: MEMO TABLE

	Best Expr	Cost
{ABC}		
{AB}		
{A}		
{B}		
{C}		

Output:	Logical Exprs	Physical Exprs
{ABC}	1. {AB} > {C}	
Properties:	None	

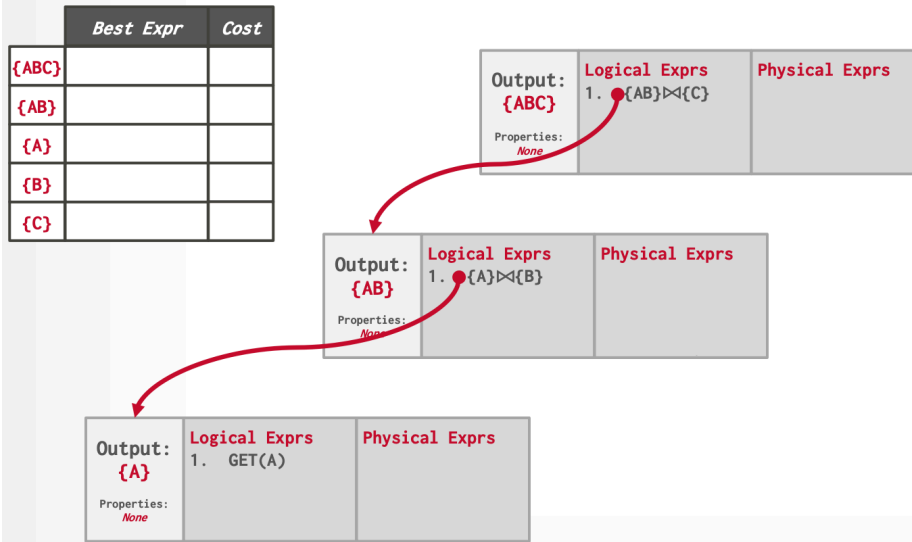
CASCADES: MEMO TABLE

	Best Expr	Cost
{ABC}		
{AB}		
{A}		
{B}		
{C}		

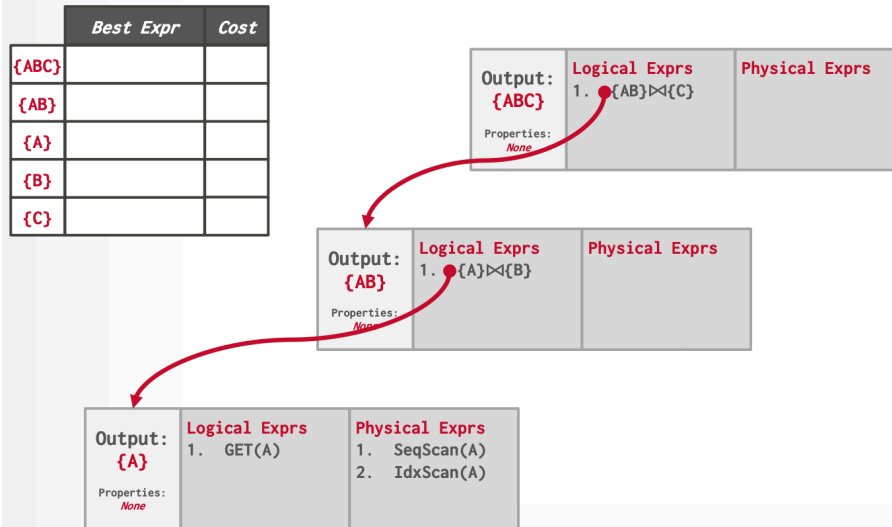
Output:	Logical Exprs	Physical Exprs
{ABC}	1. {AB} > {C}	
Properties:	None	

Output:	Logical Exprs	Physical Exprs
{AB}	1. {A} > {B}	
Properties:	None	

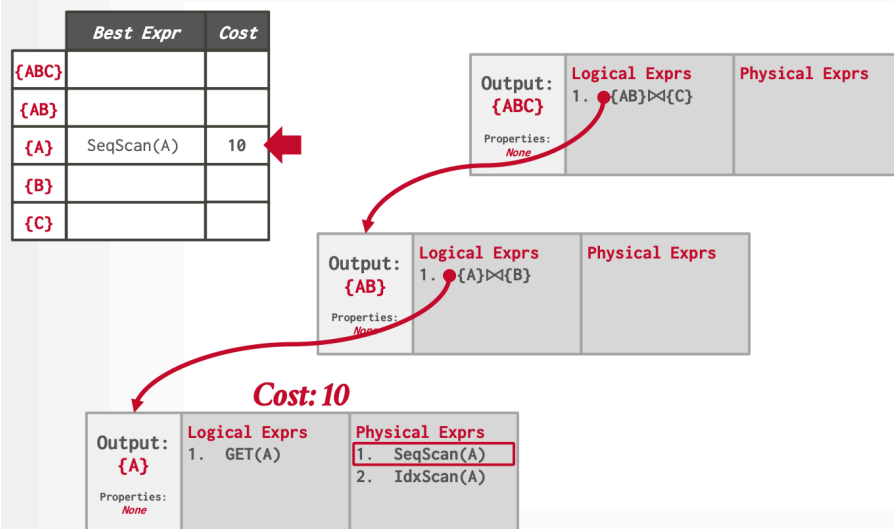
CASCADES: MEMO TABLE



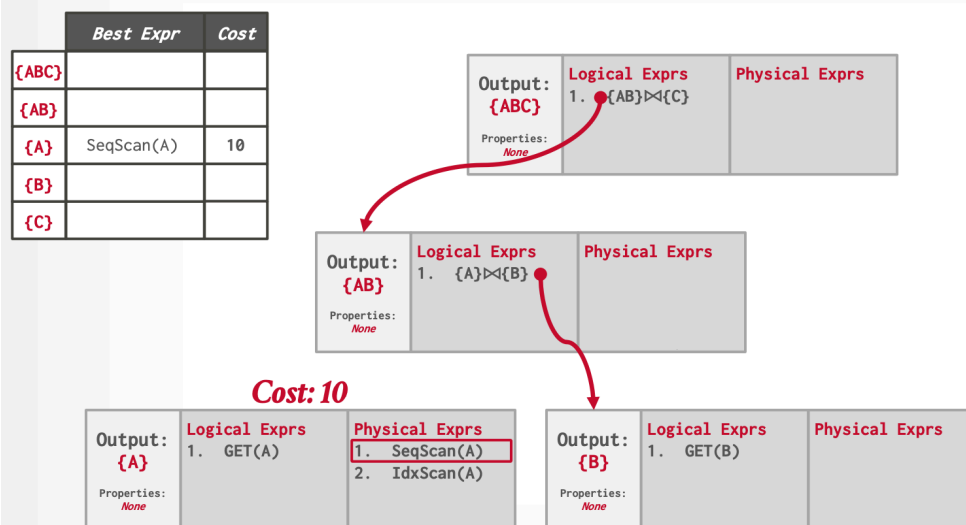
CASCADES: MEMO TABLE



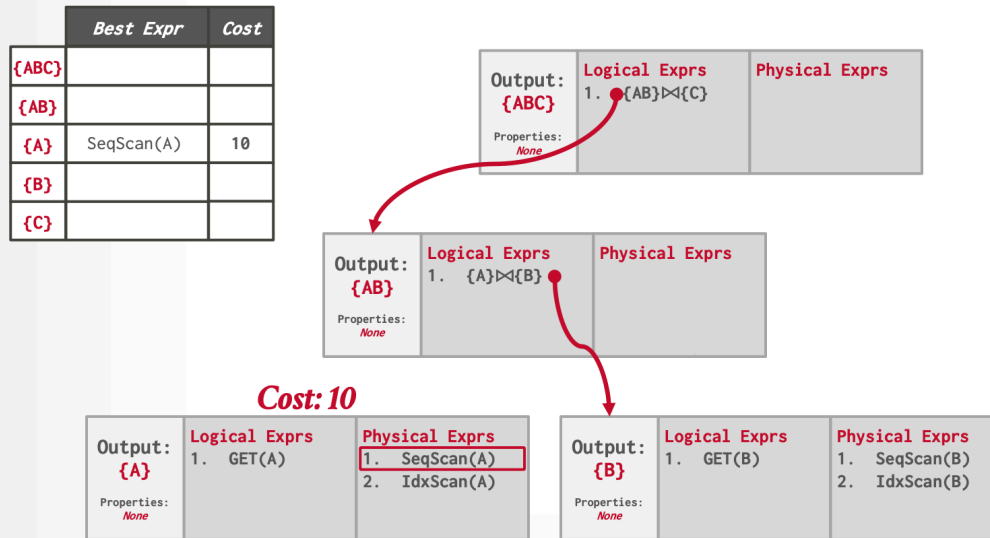
CASCADES: MEMO TABLE



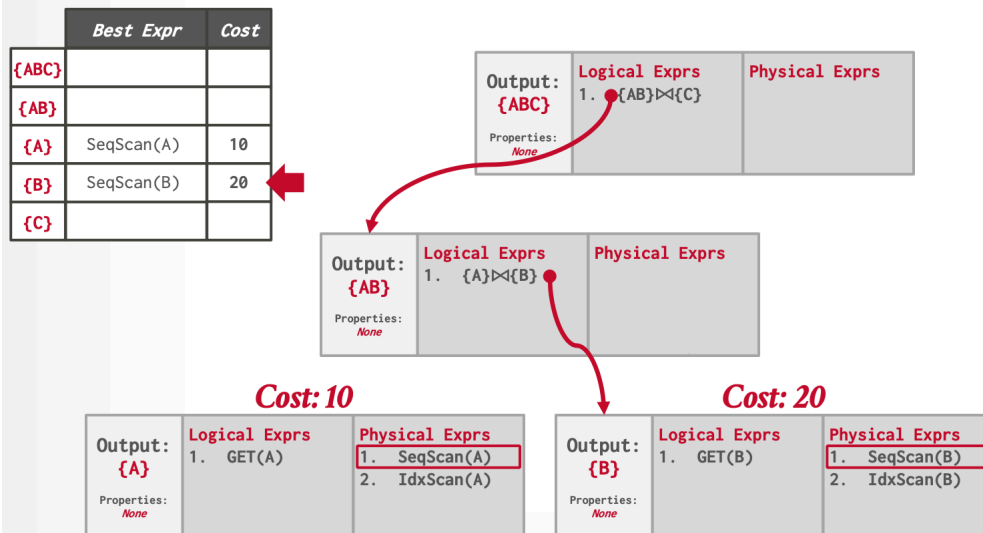
CASCADES: MEMO TABLE



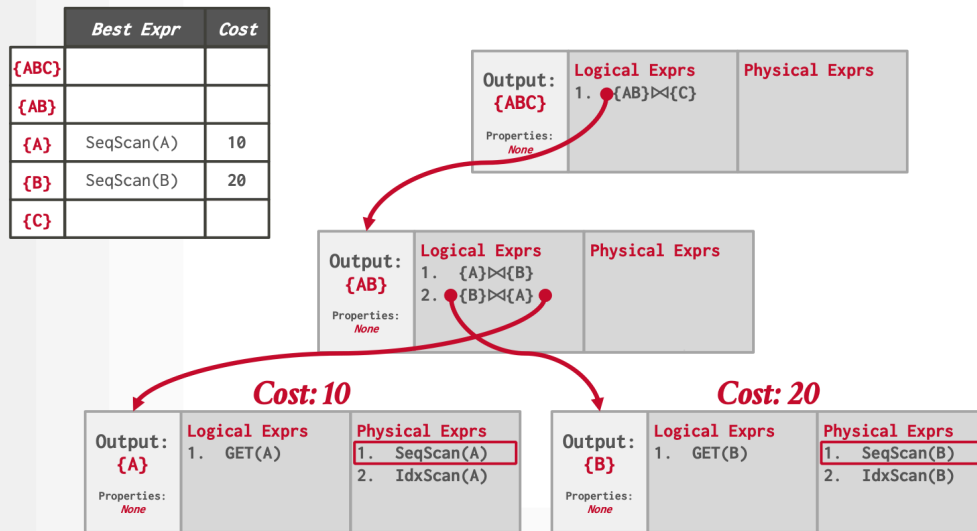
CASCADES: MEMO TABLE



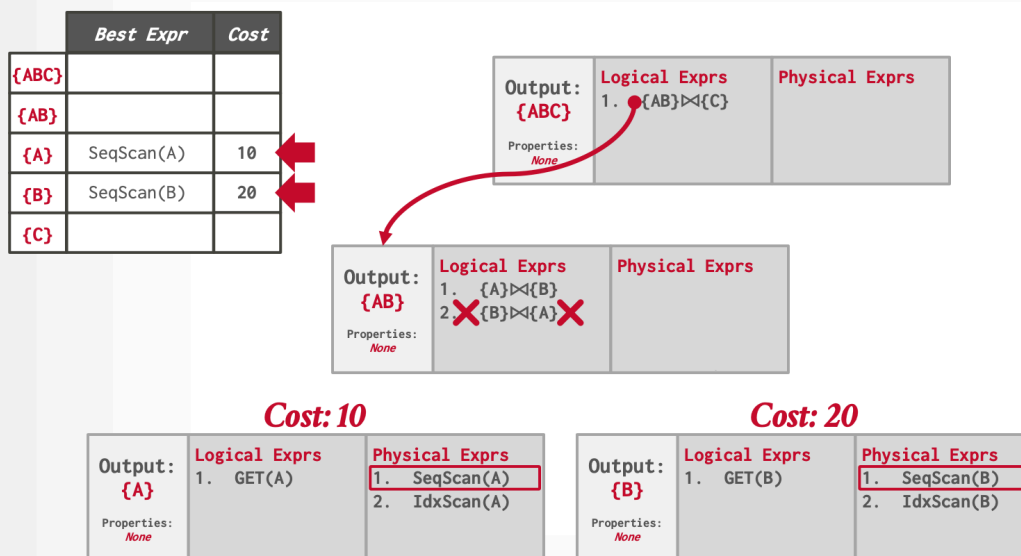
CASCADES: MEMO TABLE



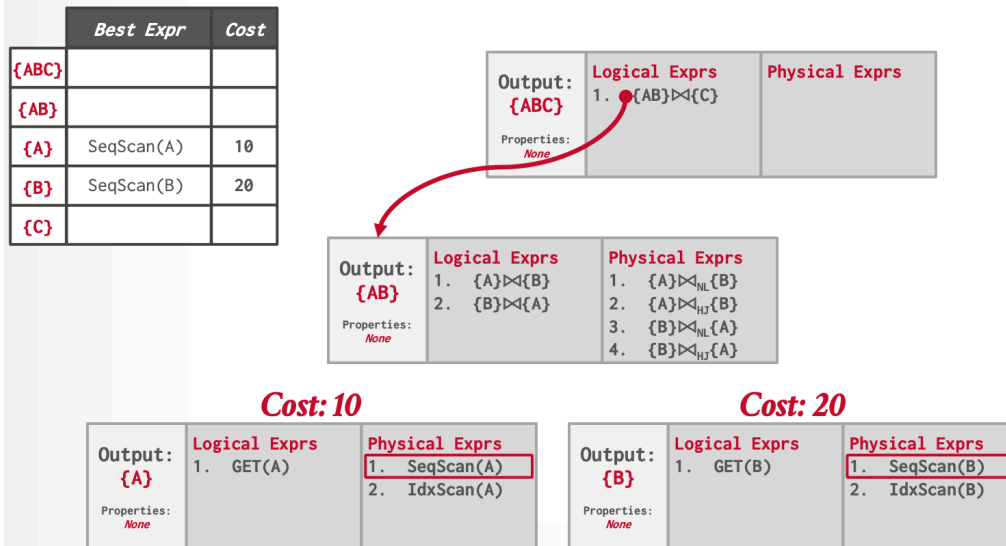
CASCADES: MEMO TABLE



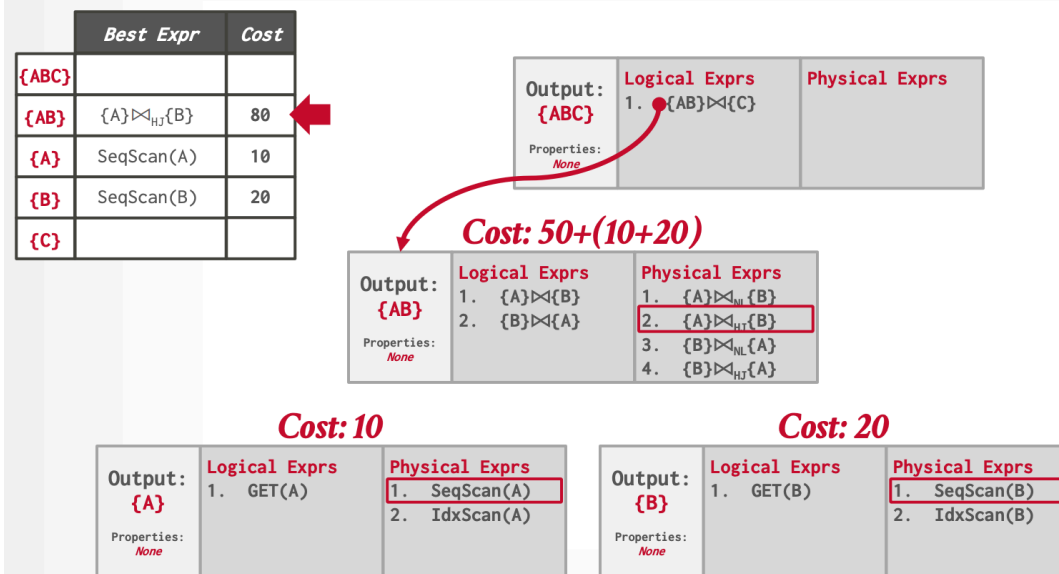
CASCADES: MEMO TABLE



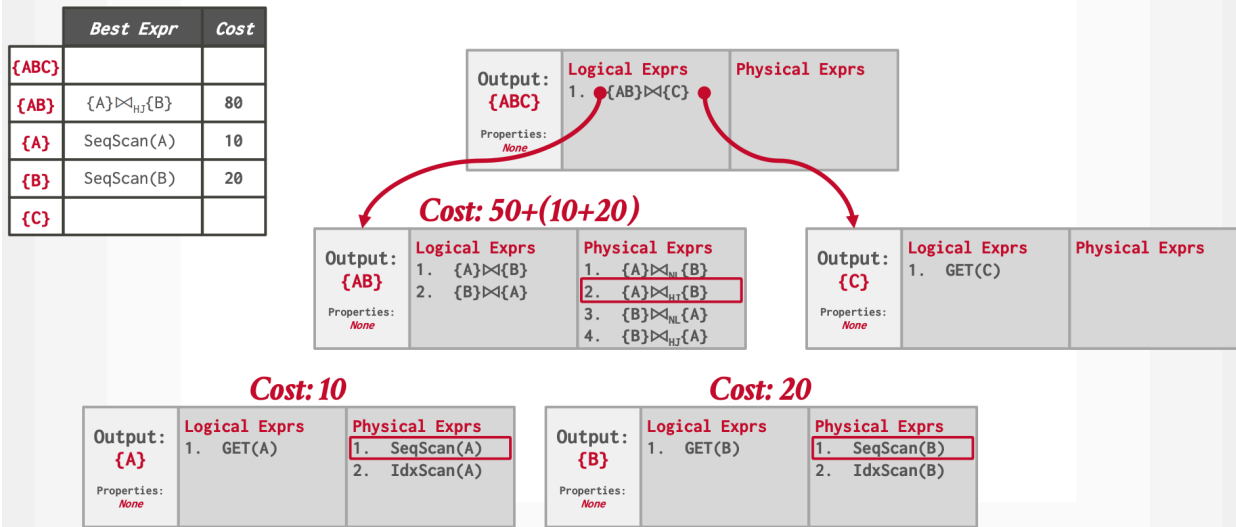
CASCADES: MEMO TABLE



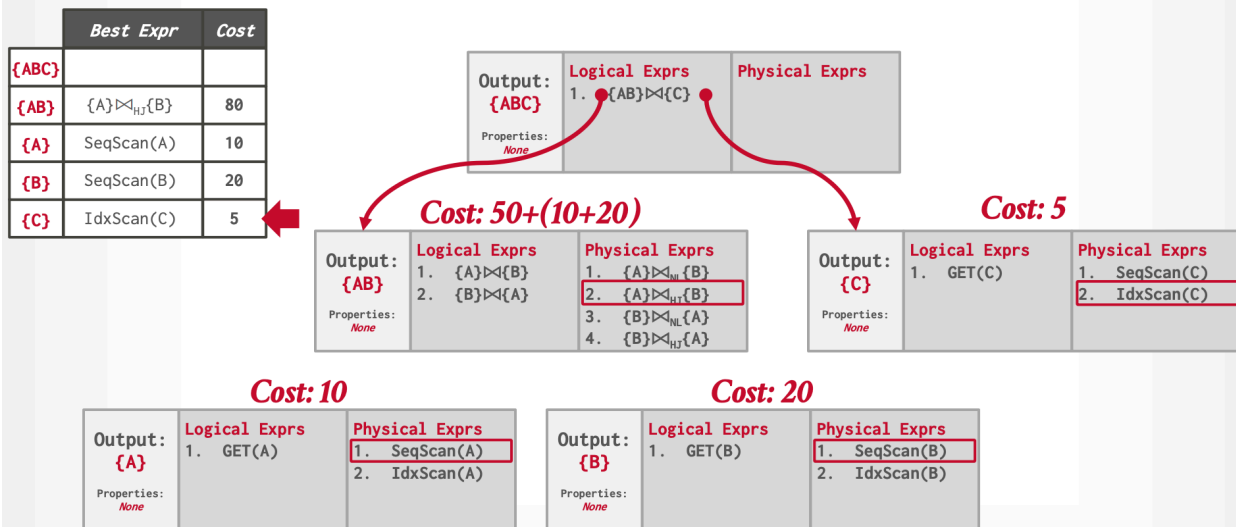
CASCADES: MEMO TABLE



CASCADES: MEMO TABLE



CASCADES: MEMO TABLE



CASCADES: MEMO TABLE

	Best Expr	Cost
{ABC}		
{AB}	{A} ⋈ _{HJ} {B}	80
{A}	SeqScan(A)	10
{B}	SeqScan(B)	20
{C}	IdxScan(C)	5

Output: {ABC}	Logical Exprs 1. {AB} ⋈ _I {C} 2. {BC} ⋈ _I {A} 3. {AC} ⋈ _I {B} 4. {B} ⋈ _I {AC}	Physical Exprs 1. {AB} ⋈ _{HL} C 2. {BC} ⋈ _{HL} A 3. {AC} ⋈ _{HL} B ⋮
Properties: <i>None</i>		

Cost: 50+(10+20)

Cost: 5

Output: {AB}	Logical Exprs 1. {A} ⋈ _I {B} 2. {B} ⋈ _I {A}	Physical Exprs 1. {A} ⋈ _{HL} {B} 2. {A} ⋈ _{HL} {B} (highlighted) 3. {B} ⋈ _{HL} {A} 4. {B} ⋈ _{HL} {A}
Properties: <i>None</i>		

Output: {C}	Logical Exprs 1. GET(C)	Physical Exprs 1. SeqScan(C) 2. IdxScan(C) (highlighted)
Properties: <i>None</i>		

Cost: 10

Cost: 20

Output: {A}	Logical Exprs 1. GET(A)	Physical Exprs 1. SeqScan(A) (highlighted) 2. IdxScan(A)
Properties: <i>None</i>		

Output: {B}	Logical Exprs 1. GET(B)	Physical Exprs 1. SeqScan(B) (highlighted) 2. IdxScan(B)
Properties: <i>None</i>		

SPECIAL TOPICS (SPRING 2025)

CASCADES: MEMO TABLE

	Best Expr	Cost
{ABC}	((A) ⋈ _{HJ} (B)) ⋈ _{HJ} {C}	125
{AB}	{A} ⋈ _{HJ} {B}	80
{A}	SeqScan(A)	10
{B}	SeqScan(B)	20
{C}	IdxScan(C)	5

Cost: 40+(80+5)

Output: {ABC}	Logical Exprs 1. {AB} ⋈ _I {C} 2. {BC} ⋈ _I {A} 3. {AC} ⋈ _I {B} 4. {B} ⋈ _I {AC}	Physical Exprs 1. {AB} ⋈ _{HL} C 2. {BC} ⋈ _{HL} A 3. {AC} ⋈ _{HL} B ⋮
Properties: <i>None</i>		

Cost: 50+(10+20)

Cost: 5

Output: {AB}	Logical Exprs 1. {A} ⋈ _I {B} 2. {B} ⋈ _I {A}	Physical Exprs 1. {A} ⋈ _{HL} {B} 2. {A} ⋈ _{HL} {B} (highlighted) 3. {B} ⋈ _{HL} {A} 4. {B} ⋈ _{HL} {A}
Properties: <i>None</i>		

Output: {C}	Logical Exprs 1. GET(C)	Physical Exprs 1. SeqScan(C) 2. IdxScan(C) (highlighted)
Properties: <i>None</i>		

Cost: 10

Cost: 20

Output: {A}	Logical Exprs 1. GET(A)	Physical Exprs 1. SeqScan(A) (highlighted) 2. IdxScan(A)
Properties: <i>None</i>		

Output: {B}	Logical Exprs 1. GET(B)	Physical Exprs 1. SeqScan(B) (highlighted) 2. IdxScan(B)
Properties: <i>None</i>		

SPECIAL TOPICS (SPRING 2025)

OBSERVATION

Promises enable Cascades to potentially target more beneficial transformations more quickly in the search process.

Cascades' flexible architecture enables other optimizations to reduce search times.

SIMPLIFICATION RULES

Some rules simplify the logical query plan and almost always reduce its cost.

Instead of creating alternative expressions, a **simplification rule** replaces the expression with the transformed expression.

- Removes the need to retain unnecessary state.
- Equivalent to Starburst rewrite rules.

MACRO RULES

To reduce the complexity of the search space and find a better plan more quickly, the optimizer can support **macro rules** that apply multiple transformations in a single rule.

This goes against the spirit of simple, independent rules as defined in the Volcano paper.

PARALLEL SEARCH

If tasks are independent, then the optimizer can execute them in parallel on multiple threads.

- Memo table is the shared state of the optimization process.
- Need to ensure internal data structures are thread-safe.

AFAIK, [Orca](#) is the only multi-threaded Cascades optimizer implementation.

We will discuss this more in [Lecture #17](#).

CASCADES IMPLEMENTATIONS

Standalone:

- [Wisconsin OPT++](#) (1990s)
- [Portland State Columbia](#) (1990s)
- [Greenplum Orca](#) (2010s)
- [CMU optd](#) (2025)

Integrated:

- [Microsoft SQL Server](#) (1990s)
- [Tandem NonStop SQL](#) (1990s)
- [Clustrix](#) (2000s)
- [CockroachDB](#) (2010s)
- [Snowflake](#) (2010s)
- [Databricks](#) (2010s)

MICROSOFT SQL SERVER

First Cascades implementation started in 1995.

- Derivatives are used in many MSFT database products.
- All transformations are written in C++. No DSL.
- Scalar / expression transformations are written in procedural code and not rules.

DBMS applies transformations in multiple stages with increasing scope and complexity.

- The goal is to leverage domain knowledge to apply transformations that you always want to do first to reduce the search space.

MSSQL: MULTI-STAGE OPTIMIZATION

Sub-Query Removal
Outer Joins to Inner Joins
Predicate Pushdown
Empty Result Pruning

**Simplification /
Normalization**

*Tree-to-Tree
Transformations*

*Cost-based Search
Initialization*

Pre-Exploration

Trivial Plan Short-circuit
Projection Normalization
Statistics Identification/Collection
Initial Cardinality Estimates
Join Collapsing

Stage1: Trivial Plan
Stage2: Quick Plan (Parallel)
Stage3: Full Plan (Parallel)

Exploration

*Multi-Stage
Cost-Based Search*

*Engine-Specific
Transformations*

Post-Optimization

MSSQL: OPTIMIZATIONS

Optimization #1: Timeouts are based on the number of transformations not wallclock time.

→ Ensures that overloaded systems do not generate different plans than under normal operations.

Optimization #2: Pre-populate the Memo Table with potentially useful join orderings.

→ Heuristics that consider relationships between tables.

→ Syntactic appearance in query.

GREENPLUM ORCA

Standalone Cascades implementation in C++.

- Originally written for Greenplum.
- Extended to support HAWQ.
- Supports multi-threaded search.

A DBMS integrates Orca by implementing API to send catalog + stats + logical plans and then retrieve physical plans.

Open-sourced in 2010s but then closed-sourced in 2024 after Broadcom acquisition.

ORCA: A MODULAR QUERY OPTIMIZER
ARCHITECTURE FOR BIG DATA
IGMOD 2014

COCKROACHDB

Custom Cascades implementation written in 2018.

All transformation rules are written in a custom DSL (OptGen) and then codegen into Go-lang.

- Can embed Go logic in rule to perform more complex analysis and modifications.

Also considers scalar expression (predicates) transformations together with relational operators.

PARTING THOUGHTS

Andy prefers the Cascades' architecture because it is conceptually cleaner than other implementations.

→ The quality of a plan is still highly dependent on accurate statistics and cost estimations.

The Germans disagree with top-down search...